Egyptian Informatics Journal 29 (2025) 100638

Contents lists available at ScienceDirect

Egyptian Informatics Journal

journal homepage: www.sciencedirect.com

Full Length Article

A hybrid GRASP and VND heuristic for vehicle routing problem with dynamic requests

Shifeng Chen [a,*], Yanlan Yin [b], Haitao Sang [a], Wu Deng [c]

[a] School of Electronic and Electrical Engineering, Lingnan Normal University, Zhanjiang 524048, China
[b] School of Computer and Intelligence Education, Lingnan Normal University, Zhanjiang 524048, China
[c] College of Electronic Information and Automation, Civil Aviation University of China, Tianjin 300300, China

ARTICLE INFO

Keywords:
VRP
Dynamic requests
GRASP
VND

ABSTRACT

This paper describes a hybrid heuristic that integrates the Greedy Randomized Adaptive Search Procedure (GRASP) and Variable Neighborhood Descent (VND) to address the Vehicle Routing Problem with Dynamic Requests (VRPDR). The VRPDR, a dynamic offshoot of the classical Vehicle Routing Problem (VRP), features customer requests emerging over time, with the objective of minimizing the total travel distance by devising a set of routes to serve all customers. The proposed method initially employs GRASP to construct an initial solution, followed by VND for exploration and refinement. The hybrid approach aims to utilize the strengths of both algorithms. Through testing on two sets of benchmark instances, namely dynamic pickup instances and dynamic delivery instances, 15 new optimal solutions are identified for the former and 11 for the latter. These results clearly demonstrate that the proposed algorithm competes favorably with the algorithms documented in the literature.

1. Introduction

The traditional vehicle routing problem (VRP) is a frequently employed optimization problem in many areas such as logistics, transportation, and distribution. It was originally proposed by Dantzig and Ramser [1] and is derived from the travelling salesman problem (TSP). The aim of the VRP is to determine the most efficient routes for a fleet of vehicles to visit a set of customers and return to the depot while adhering to specific constraints. Over time, several variants of the VRP have been developed to mimic real-world situations, such as the VRP with time windows [2,3], multi-depot VRP [4,5], time-dependent VRP [6,7], Electric VRP [8,9], Green VRP [10,11], and Dynamic vehicle routing problem (DVRP) [12–14], among others. Among these variants, DVRP has become a particularly popular research issue due to its closer alignment with real-life logistics scenarios.

The Vehicle Routing Problem with Dynamic Requests (VRPDR) is a practical variation of the classic VRP that incorporates dynamic customer demands. In comparison to the traditional VRP, VRPDR takes into account new orders that may arise during operations, making it particularly suitable for e-commerce logistics and related industries. Within the realm of logistics and distribution optimization, VRPDR plays a pivotal role in promptly responding to order changes and optimizing routes, thereby reducing distribution costs, enhancing corporate profitability and market competitiveness, improving customer satisfaction, as well as supporting the intelligent transformation of logistics enterprises. Consequently, delving into VRPDR and its solution algorithms holds both academic significance and significant practical application.

Several methods have been effectively used to solve the VRP and its variants, and many of these can be applied to the VRPDR as well. However, the VRPDR poses unique challenges due to its real-time adaptation requirements in response to changing customer demands. This characteristic distinguishes the VRPDR from the traditional VRP, where customer demands are typically known and fixed beforehand. To effectively address the VRPDR, methods must incorporate flexible and dynamic route planning algorithms that can adapt to new requests. Psaraftis [15] first introduced the concept of periodic reoptimization to address dynamic dial-a-ride problem. Scenarios for dynamic VRP were presented by Kilby et al. [16], while Montemanni et al. [17] conducted a simulation of the dynamic VRP. With this context, the concept of a working day, denoted as T, was introduced. In this process, T is divided into n equal-length time slices. Within each time slice, a static VRP is formulated and solved using an ant colony system. After that, several solution methods have been proposed. Mostepha et al. [18] developed an approach that combine particle swarm optimization (PSO) and

* Corresponding author.
E-mail address: chens@lingnan.edu.cn (S. Chen).

https://doi.org/10.1016/j.eij.2025.100638
Received 17 July 2024; Received in revised form 16 February 2025; Accepted 28 February 2025
Available online 5 March 2025

variable neighborhood search (VNS) for VRPDR, assessing its dynamic adaptability. Mańdziuk et al. [19] implemented a memetic algorithms to address the VRPDR. Yi et al. [20] introduced a hybrid genetic algorithm (GA) specifically designed for the VRPDR. The GA incorporates a greedy split, a local search based on adjacent-exchange, and a novel mutation operator called insert mutation. Furthermore, Abdirad [21] et al. presented a two-stage hybrid algorithm to solve the DVRP. This approach encompasses construction algorithms to form the initial routes, and improvement algorithms to refine them. Lan et al. [22] addressed the dynamic dispatch waves problem, wherein vehicles are dispatched at fixed decision moments. At each decision moment, they propose an iterative conditional dispatch algorithm to determine which known requests to dispatch and the most efficient routing for these dispatched requests. Additionally, Zhou et al. [23] formulated a memetic algorithm (MA) to tackle the dynamic pickup and delivery problem challenges encountered in the real-world scenarios. Their approach also employs time slices, meaning that orders are periodically released at designated time intervals. When new orders are released at time t, all uncompleted orders are first organized to form a static Pickup and Delivery Problem. The MA is then utilized to solve this static challenge, ultimately constructing a routing plan. A similar approach was also adopted by Abdallah et al. [24] and Ren et al. [25].

In recent years, the combination of Greedy Randomized Adaptive Search Procedure (GRASP) and Variable Neighborhood Descent (VND) has emerged as a powerful heuristic approach for solving complex optimization problems, particularly the Vehicle Routing Problem (VRP) and its variants. This hybrid metaheuristic algorithm leverages the strengths of both GRASP and VND to efficiently explore the solution space and find near-optimal solutions for these challenging combinatorial problems. Gil-Borrás et al. [26] presented a novel algorithm that combines GRASP and VND to tackle the online order batching problem. This approach leverages GRASP to generate efficient starting points for VND, which then refines the solutions. The proposed method surpasses previous state-of-the-art attempts in terms of performance. Lucía Barrero et al. [27] implemented a GRASP/VND specifically designed for the heterogeneous fleet vehicle routing problem with time windows. This implementation incorporates five distinct local searches and demonstrates superior performance compared to the exact solver CPLEX, particularly in terms of computational time and solution quality for large-scale instances. Débora P. Ronconi [28] et al. proposed GRASP and VNS approaches for a vehicle routing problem with step cost functions. Their computational analysis of literature and benchmark instances based on real-world scenarios shows that these methods are highly suitable for practical applications and exhibit strong performance in various situations. Nikolaos A. Kyriakakis et al. [29] introduced the Energy Minimizing Drone Routing Problem with Pickups and Deliveries, aimed at addressing direct trading scenarios between individuals using drones for courier services. They formulated the problem and implemented three variants of the GRASP metaheuristic to solve it. The algorithms were compared using generated benchmark instances, focusing on both energy and distance objectives.

This paper presents a hybrid heuristic that integrates the Greedy Randomized Adaptive Search Procedure (GRASP) and Variable Neighborhood Descent (VND) to address the Vehicle Routing Problem with Dynamic Requests (VRPDR). The VRPDR, a complex variant of the classic VRP, is characterized by the dynamic appearance of customer requests over time. The algorithm initiates with GRASP to construct the initial solution, which is then succeeded by VND to explore new search areas and refine the identified solution, ultimately improving its quality. This hybrid approach utilizes the complementary advantages of GRASP and VND to effectively navigate the complex solution space of VRPDR. Our contribution is fourfold:

We devise a novel hybrid approach for VRPDR that combines an iterative construction process from GRASP with a diversified local search from VND. This integration aims to capitalize on the rapid convergence of GRASP and the thorough exploration capabilities of VND.

We have specifically designed four greedy construction algorithms. These algorithms are tailored to address the inherent challenges of dynamic request scenarios, providing a robust foundation for the initial solution development.

We develop a series of neighborhood structures based on the characteristics of VRPD. These structures not only efficiently handle the constraints of static VRP, but also effectively respond to the emergence and changes of dynamic requests, thereby finding higher-quality feasible solutions within the solution space.

We conduct an extensive experimental evaluation on benchmark instances, demonstrating that our proposed approach achieves superior performance compared to existing methods in the literature. The results highlight the efficacy of our approach in yielding high-quality solutions within reasonable computational times.

The remainder of this paper is structured as follows. Section 2 defines the VRPDR and presents its mathematical formulation. In Section 3, we detail our hybrid algorithmic approach, which combines the GRASP and VND to effectively tackle the VRPDR. Section 4 reports and analyzes the experimental results obtained from implementing our proposed method. Finally, Section 5 summarizes the conclusions drawn from this study and outlines potential directions for future research.

2. Problem definition and notations

The VRPDR is defined by an undirected graph $G = (V, E)$, with $V = \{0, 1, \ldots, n\}$ representing vertices, and $E = \{(i, j) \mid i, j \in V, i \neq j\}$ denoting edges connecting each pair of vertices. Vertex 0 represents the depot, while the remaining vertices, $C = V \setminus \{0\} = \{1, 2, \ldots, n\}$ represent n customers. The cost of traveling from v_i to v_j denoted by c_{ij}, which typically corresponds to the Euclidean distance d_{ij} or travel time t_{ij} between them. The depot has a specific opening time e_0 and closing time l_0 ($0 \leq e_0 < l_0$). A set K of homogeneous vehicles with a maximum capacity Q is available in depot, and each vehicle must return to the depot after service all customers. Additionally, each customer i is associated with a location (x_i, y_i), an appearance moment τ_i, a demand q_i, and a service duration s_i. If a customer i is static, its available time τ_i is -1; otherwise, it falls within the working day ($e_0 \leq \tau_i \leq l_0$). Furthermore, each customer can only be served by one car, and only once. A feasible solution to the VRPDR can be expressed as $S = \{r_0, r_1, \ldots, r_k\}$, where $r_i = \{v_0, v_1, \ldots, v_p, v_{p+1}\}$ represents a route that starts and ends at the depot ($v_0 = v_{p+1} = 0$) and includes p customers.

To enhance understanding of the VRPDR, Fig. 1 illustrates a VRPDR scenario. Initially at time t_0, two routes (vehicles) were scheduled to serve seven customers based on their requests. However, While the vehicles were in transit, two new customer requests appear at time t_1, as a result, the depot had to re-plans the routes to hold the remaining unserved customers. It is important to note that while a vehicle is driving on the road, it cannot change its route arbitrarily as it may violate traffic rules. Therefore, when a dynamic customer requests, one must wait for the vehicle to reach the nearest customer's location before re-optimize the route. In this paper, we refer to the vehicle that arrives at the nearest customer as a virtual depot. Finally, at time t_2, all customers were served, and the vehicles returned to the depot.

Base on the idea presented in [16], to accommodate dynamic requests, the algorithm implements a cutoff time T_{co}. All new orders received before T_{co} are accepted and processed within the current working day, while orders received after T_{co} are postponed to the nest day. As depicted in Fig. 2, the working day T is then further divided into n_{ts} equal-length time slices, each represents a partial static VRP. In the first time slice only consider customers know from the previous working day. Subsequent time slices will account for new customers received in the previous time slice as well as those who have not yet been visited by drivers.

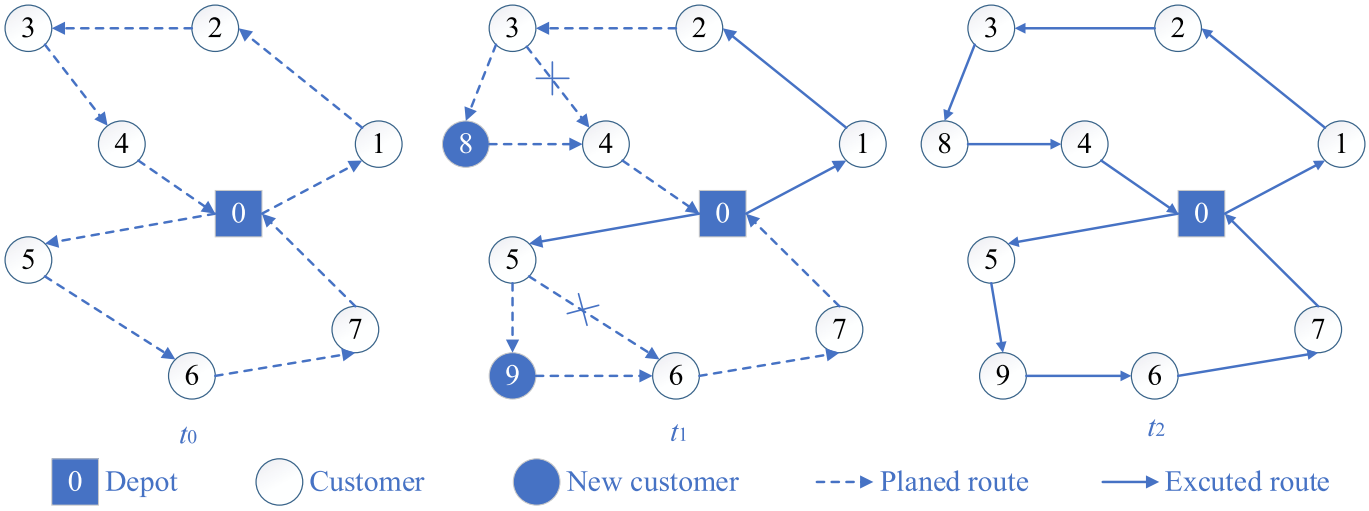

Fig. 1. An example of VRPDR.

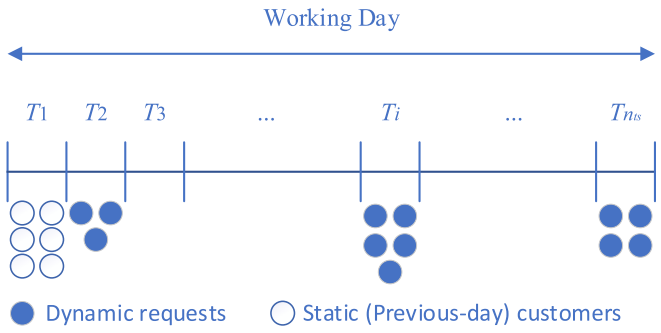

Fig. 2. The division of the working day.

The variables are as follows: $x_{ij}^k \in \{0,1\}$, where a value of 1 represents the edge (i,j) is travelled by vehicle k and 0 otherwise; $a_i \geq 0$ which represents the arrival time of a vehicle to customer i. The objective is to find a set of vehicle routes that minimize the total traveling cost, this can be formulated as follows:

$$f(x) = \min \sum_{(i,j)\in E} \sum_{k\in K} c_{ij} \bullet x_{ij}^k \tag{1}$$

s.t.

$$\sum_{i\in V} x_{ij}^k = \sum_{i\in V} x_{ji}^k \in \{0,1\}, j \in C, k \in K, i \neq j \tag{2}$$

$$\sum_{k\in K} \sum_{j\in V} x_{ij}^k = 1, i \in C, i \neq j \tag{3}$$

$$\sum_{j\in V} x_{0j}^k = \sum_{i\in V} x_{i0}^k \leq K \tag{4}$$

$$\sum_{i\in V} \sum_{j\in V} q_i \bullet x_{ij}^k \leq Q, k \in K \tag{5}$$

$$a_i = \begin{cases} 0, i = 0 \\ a_{i-1} + s_{i-1} + t_{i-1,i}, 1 \leq i \leq l+1 \end{cases} \tag{6}$$

$$e_0 \leq a_i \leq l_0 \tag{7}$$

$$x_{ij}^k \in \{0,1\} \tag{8}$$

Eq. (1) denotes the objective function of the VRPDR, which represent minimize the travel distance. Constraint (2) is a flow conservation constraint, ensuring that each customer j has either zero or one in-degree and out-degree. Constraint (3) ensures that each customer i ($i \in C$) is visited by exactly one vehicle. Constraint (4) stipulates that every route starts and ends the depot. Constraint (5) specifies the capacity of each vehicle. Constraint (6) defines arrival time for each customer. Constraint (7) restricts that each customer is only served during the working hours. Finally, Constraint (8) imposes restrictions on the decision variables.

3. Solution methodology

A hybrid algorithm is a technique that combines metaheuristics with other optimization techniques (such as exact algorithms, heuristics, or metaheuristics) to improve performance. The combination of multiple strategies is expected to take advantage of each technique's strengths, resulting in better performance. To deal with the VRPDR, we propose a hybrid algorithm based on the Greedy Randomized Adaptive Search Procedure (GRASP) and the Variable Neighborhood Descent (VND). The GRASP, introduced by Feo and Resende [30] in 1989, is a powerful multi-start process that consists of two phases: construction and improvement. During the construction phase an initial feasible solution is generated randomly using a greedy function. In the improvement phase, a local search algorithm is employed to attempts to improve the solution within the neighborhood of the initial solution. On the other hand, the VND, proposed by Mladenovic and Hansen [31] in 1997, is a variant of the variable neighborhood search algorithm. it employs a range of neighborhood structures to enhance the current solution. This is because different neighborhood structures do not always have the same local minima. Therefore, the obtained solution must be the local optimal solution under all neighborhood structures. Hence, VND is utilized as a local search method for GRASP in this paper.

The pseudocode of the hybrid GRASP/VND metaheuristic for the VRPDR is presented in Algorithm 1. The algorithm starts by initializing the best-found solution S^* to null and its objective function $f(S^*)$ to infinity (Line 1). Subsequently, iterations take place between Lines 3 and 9. In Line 4, an initial feasible solution is attempted to be constructed through a greedy search. The local optimal solution is then improved using the VND technique in Line 5. If the local optimum S_{impr} is superior to the incumbent, the best-found solution overall S^* is saved (Lines 6–8). The whole process is repeated until the maximum number of iterations (*maxIter*) is reached. Finally, the optimal solution found throughout all iterations is returned as the output of the algorithm (Line 10).

In order to handle the dynamic problem, the day is divided into equal-length time periods. At the start of each period, an instance for the

static problem is created by considering the current sets. When the new demand(s) revealed, the solution S^* is adapted as following: the last visited customer becomes the depot (starting point), while the remaining customers and any new received request(s) are rearranged to form the new routes that satisfy the system constraints.

Algorithm 1: Hybrid_GRASP_VND

Input: A maximum number of iterations *maxIter*
Output: best solution found S^*
$S^* \leftarrow \emptyset$; $f(S^*) \leftarrow \infty$
$iter \leftarrow 1$
while $iter \leq maxIter$ **do**
  $S_{init} \leftarrow$ GreedyRandomGenerator()
  $S_{impr} \leftarrow$ VND(S_{init})
  if $f(S_{impr}) < f(S^*)$ **then**
    $S^* \leftarrow S_{impr}$
  end if
end while
return S^*

The following subsections explain each phase in the GRASP algorithm in detail.

3.1. Construction phase

The construction heuristic is a method that incrementally builds a solution in the problem space by following predefined rules, The process continues until a complete solution is constructed. In our case, the construction phase involves adding one customer at a time to the current solution. If the current solution cannot accommodate any more customers, a new route is created. This process continues until all customers have been served.

Algorithm 2: Closest

Input: A solution S, a set *custs* of unvisited customers
Output: A set CL of candidate moves
$CL \leftarrow \emptyset$
foreach $c \in custs$ **do**
  foreach $r \in S$ **do**
    $p \leftarrow |r|$-2
    $u \leftarrow r[p]$
    $g \leftarrow d_{uc}$
    $CL \leftarrow CL \cup \{$ move(r, i, c, g) $\}$
  end foreach
end foreach
return CL

In the remaining of this subsection, some constructive heuristics are presented. These heuristics typically take in a partial solution S and a subset of unvisited customers $cust \subset N$ as inputs, and return a set of candidate moves CL. Each element $m \in CL$ is an ordered quaternion $m =$ move (r, i, c, g) where $g \in R$ represents the cost of inserting customer c in the i-th position of route r.

The **Closest** heuristic is an algorithm that generates a list of candidates by inserting a customer $c \in cust$ at the end of a route r. The associated cost g represents the distance between the current customer c and the last customer u on route r. The detailed pseudocode description is presented in Algorithm 2.

Algorithm 3: Earliest

Input: A solution S, a set *custs* of unvisited customers
Output: A set CL of candidate moves
$CL \leftarrow \emptyset$
foreach $r \in S$ **do**
  for $i = 1$ to $|r|$ **do**
    foreach $c \in custs$ **do**
      $g \leftarrow a_i' - a_i$
      $CL \leftarrow CL \cup \{$ move(r, i, c, g) $\}$
    end foreach
  end for
end foreach
return CL

The **Earliest** heuristic, as presented in Algorithm 3, generates a candidate list representing the insertion of customer $c \in cust$ at any position within route r. The associated cost g is the delay of the arrival time of the customer i after inserting customer c.

Algorithm 4: Farthest

Input: A solution S, a set *custs* of unvisited customers
Output: A set CL of candidate moves
$CL \leftarrow \emptyset$
foreach $r \in S$ **do**
  for $i = 1$ to $|r|$ **do**
    foreach $c \in custs$ **do**
      $u \leftarrow r[i-1]$
      $v \leftarrow r[i]$
      $g \leftarrow d_{uv} - d_{uc} - d_{cv}$
      $CL \leftarrow CL \cup \{$ move(r, i, c, g) $\}$
    end foreach
  end for
end foreach
return CL

The **Farthest** heuristic is an algorithm that generates a candidate list by inserting a customer $c \in cust$ into the position that maximizes cost. The cost g is defined as the extra travel distance caused by insertion of customer c. Algorithm 4 describes its pseudocode.

Algorithm 5: Nearest

Input: A solution S, a set *custs* of unvisited customers
Output: A set CL of candidate moves
$CL \leftarrow \emptyset$
foreach $r \in S$ **do**
  for $i = 1$ to $|r|$ **do**
    foreach $c \in custs$ **do**
      $u \leftarrow r[i-1]$
      $v \leftarrow r[i]$
      $g \leftarrow d_{uc} + d_{cv} - d_{uv}$
      $CL \leftarrow CL \cup \{$ move(r, i, c, g) $\}$
    end foreach
  end for
end foreach
return CL

In contrast to the Farthest heuristic, the **Nearest** heuristic, as presented in Algorithm 5, generates a candidate list representing the insertion of customer $c \in cust$ at the position that minimizes the additional travel distance.

Algorithm 6: GreedyRandomGenerator()

Input: a set of unvisited customers *custs*,
a random number $\alpha \in (0,1)$
Output: solution S
$S \leftarrow \emptyset$
$CL \leftarrow \emptyset$
  while $custs \neq \emptyset$ **do**
  if $CL = \emptyset$ **then**
    $c \leftarrow$ SelectRandom($custs$)
    $S \leftarrow S \cup \{0, c, 0\}$
    $custs \leftarrow custs \backslash \{c\}$
  end if
  $M \leftarrow$ Select at random from {Closest, Nearest, Farthest, Earliest}
  $CL \leftarrow M(S, custs)$
  if $CL = \emptyset$ **then**
    continue
  end if
  $g_{min} \leftarrow \min\{ g \mid$ move(r, i, c, g) $\in CL\}$
  $g_{max} \leftarrow \max\{ g \mid$ move(r, i, c, g) $\in CL \}$
  $threshold \leftarrow g_{min} + \alpha (g_{max} - g_{min})$
  $RCL \leftarrow \{$ move(r, i, c, g) $\in CL \mid g \leq threshold \}$
  move(r^*, i^*, c^*, g^*) $\leftarrow$ select at random from RCL
  $r^* \leftarrow r^* \cup \{c^*\}$
  $custs \leftarrow custs \backslash \{c^*\}$
end while
return S

The pseudocode for the construction phase is provided in Algorithm 6 which called *GreedyRandomGenerator*. It takes as input a set of unvisited

customers *custs* and a random value α ∈ [0, 1], aiming to select customers from the unvisited set and gradually construct a solution. Firstly, the solution *S* and the candidates list *CL* are both initialized as empty (lines 1–2). It then enters a loop that continues util all unvisited customers have been visited (line 3). In each iteration, if the *CL* is empty, a route is built by selecting a customer randomly, then inserts it into the solution and removes it from *custs* (lines 4–8). Next, the algorithm randomly chooses a heuristic to generate a candidate list, which includes potential customer selections (line 9). It's important to note that, considering relevant constraints, the *CL* may be empty. In such cases, the algorithm jumps to line 5. Otherwise, a threshold is calculated based on the maximum and minimum cost of any move in the *CL*, and a random value α (lines 14–16). This threshold is used to determine the number of the best candidate moves contained in the restricted candidate list (*RCL*) (line 17). Then, a candidate move is randomly chosen from this *RCL* (line 18) whose customer is included in the partial solution being constructed in this iteration. Finally, the customer just added to *S* is removed from *custs*.

3.2. Improvement phase

Once an initial solution has been constructed, it can be further enhanced using a local search algorithm with the aim of finding a local minimum. In this study, we employed a Variable Neighborhood Descent (VND) approach as the local search. Its pseudocode is reported in Algorithm 7. The input consists of a solution *S* and a set of neighborhoods *N*. and the output is an improved solution *S**. At first, the sequence of the neighborhoods is randomly shuffled (line 2), then, the heuristic tries to improve the current solution in the first neighborhood N_1 (line 4). In each iteration, a solution *S*' in the neighbourhood structure N_h with the smallest objective function value is found, the local search is restarted while updating *S* accordingly (lines 5–7). Subsequently, the VND randomly reshuffles the neighborhoods *N* (line 8) and starts again from the first neighborhood N_1. If a neighborhood fails to enhance the current solution, then the method proceeds to the next available neighborhood (line 10). When all structure has been applied and no further improvements are possible, the improvement phase concludes, and the final local optimal solution is returned (Line 13).

Algorithm 7: VND

Input: Solution *S*, neighborhoods *N*
Output: Improved solution *S*
1 $h \leftarrow 1$
2 Randomly shuffle *N*
3 **while** $h \leq |N|$ **do**
4 $S' \leftarrow N_h(S)$
5 **if** $f(S') < f(S)$ **then**
6 $S \leftarrow S'$
7 $h \leftarrow 1$
8 Randomly shuffle *N*
9 **else**
10 $h \leftarrow h + 1$
11 **end if**
12 **end while**
13 **return** *S*

We propose six neighborhood structures to tackle the VRPDR: relocate, swap, 2-opt, (1, λ) interchange, string-exchange, and Crossover, respectively.

Relocate. This movement picks a customer from one location to another in the same route. An example is given in Fig. 3 given two non-adjacent customers, *i* and *j*, in same route, we relocate the customer *i* to the location *j* by replacing the edges $(i-1, i)$, $(i, i+1)$ with $(i-1, i+1)$, and then delete the edge $(j, j-1)$ and connect the edges $(j-1, i)$ and (i, j).

Swap. This neighborhood swap locations between two customers in fixed route. Consider two customers, *i* and *j*, who are on the same route. To perform the swap, we replace the edges $(i-1, i)$, $(i, i+1)$ with $(i-1, j)$ and $(j, i+1)$. Similarly, we replace the edges $(j-1, j)$, $(j, j+1)$ with $(j-1, i)$ and $(i, j+1)$. It is illustrated in Fig. 4.

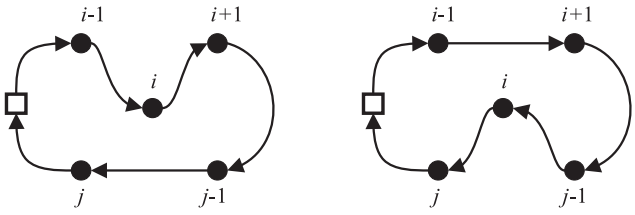

Fig. 3. Relocate.

2-opt. This operator first removes two non-adjacent edges and inverts the middle segment between, before reconnecting it with two new edges. To accomplish this, select two non-adjacent edges $(i, i+1)$ and $(j, j+1)$ from a given route where $i < j$. Substitute both links by (i, j) and $(i+1, j+1)$. Fig. 5 illustrates this move.

(1, λ) interchange. This movement involves exchanging one customer from a route with λ customers on another route. For given two node-disjoint routes, r_i and r_j, where customers *i* and *j* $(=i+\lambda)$ belong to route r_i, and a customer *k* belongs to route r_j. We interchange the segments $[i, j] \subset r_i$ with customer $k \in r_j$, as follows. First, remove edges $(k-1, k)$, $(k, k+1)$, $(i-1, i)$ and $(j, j+1)$. Then, reconnect the edges $(i-1, k)$, $(k, j+1)$, $(k-1, i)$, and $(j, k+1)$. A generic $(1, \lambda)$ interchange move is illustrated in Fig. 6.

string-exchange. This operation involves shifting a consecutive of $p \in \{1, 2\}$ customers from one route to another. The fundamental idea of string-exchange is to initially destroy two edges $(i-1, i)$ and $(j, j+1)$ from one route while simultaneously removing the edges $(k-1, k)$ and $(l, l+1)$ from another route. Subsequently, the segments $[i, j]$ and $[k, l]$ are exchanged by introducing four new edges: $(k-1, i)$, $(j, l+1)$, $(i-1, k)$ and $(l, j+1)$. This process is showed in Fig. 7.

Crossover. This movement involves selecting two customers form two different routes as exchange points, and exchanging the last parts of both routes after exchange points. Given two node-disjoint routes r_i and r_j, two customers $i \in r_i$, $j \in r_j$. This operation is completed by replacing the edges $(i, i+1)$ and $(j, j+1)$ with $(i, j+1)$ and $(j, i+1)$. The movement process is depicted in Fig. 8.

4. Experimental results

This section aims to experimentally verify the effectiveness of the proposed GRASP/VND algorithm in addressing the VRPDR. The adopted benchmarks are described in Section 4.1, Section 4.2 outlines crucial parameters of the algorithms. Section 4.3 assesses the import of the integrated components on the search performance. Finally, Section 4.4 presents computational results that compare with other methodologies reported in the scientific literature.

The source code was implemented using the C# programming language, compiled for the.NET Core 6.0 platform. All experiments were conducted on an Intel® Xeon® Bronze 3104 CPU @ 1.70 GHz processor with 16 GB RAM, and running the Windows Server 2022 Standard operating system.

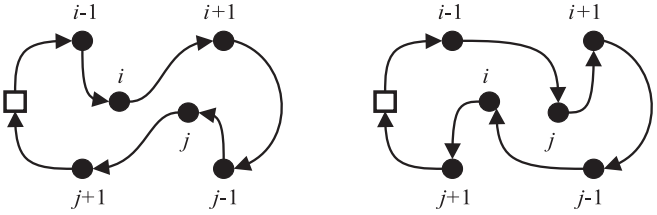

Fig. 4. 3 Swap.

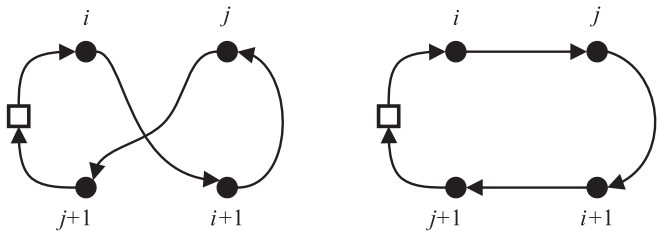

Fig. 5. 2-opt.

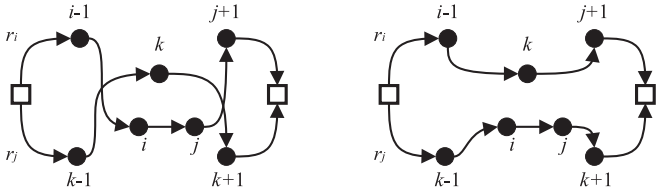

Fig. 6. (1,λ) interchange.

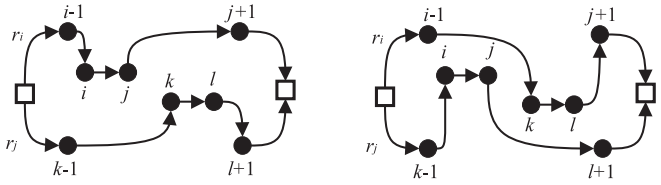

Fig. 7. String-Exchange.

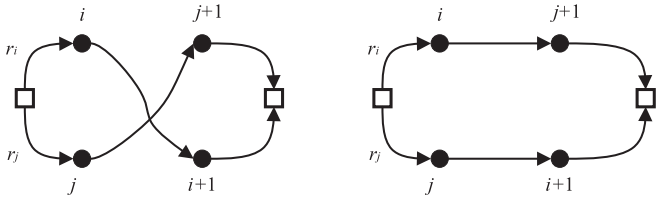

Fig. 8. Crossover.

4.1. Benchmark description

This study employs two group of examples to evaluate the performance of the proposed approach: pickup and delivery instances, each derived from three well-known capacitated vehicle routing problems (VRP). These instances comprise 12 instances from the Taillard benchmark, 7 instances from the Christofides benchmark, and 2 instances from the Fisher benchmark. The datasets are founded on the works of Kilby et al. [16] and Montenegro et al. [17]. The characteristics of the datasets are summarized in Tables 1 and 2, where N indicates the customer number, Q represents the capacity of each vehicle, T denotes the length of the working day, and S, τ correspond to the service time and available time for each customer, respectively. The datasets vary in size and customer distribution.

4.2. Parameter settings

This section is divided into two main parts. The first part discusses the parameters of the DVRP model, while the second part presents the parameter settings used in our proposed approach.

Table 3 lists the parameters of the DVRP model. Base on Kilby et al. [16] idea, the cutoff time (t_{co}) represents the time threshold after which any newly arrived customers are assumed to be the same customers as those remaining from the previous working day. The number of time slices n_{ts} determines the frequency at which new static VRP is created and solved. The commitment time (t_{ac}) indicates that a customer request must be assigned to a driver at least t_{ac} seconds prior to the planned time of departure from the last visited customer, thereby providing drivers with a suitable response time upon receiving new requests.

Montenegro et al. [17] conducted experiments and found that n_{ts}

Table 1
Characteristics of the dynamic pickup instances.

Type	Instance	N	Q	T	S	τ
Christofides	c50	50	160	351	15	[0, 345]
	c75	75	140	346	16	[0, 345]
	c100	100	200	399	12	[0, 395]
	c100b	100	200	468	12	[0, 439]
	c120	120	200	794	13	[0, 792]
	c150	150	200	399	10	[0, 398]
	c199	199	200	399	9	[0, 397]
Fisher	f71	71	30,000	211	5	[0, 208]
	f134	134	2210	11,741	13	[0, 11700]
Taillard	tai75a	75	1445	769	32	[0, 768]
	tai75b	75	1679	905	26	[0, 903]
	tai75c	75	1122	782	25	[0, 776]
	tai75d	75	1699	789	27	[0, 783]
	tai100a	100	1409	897	30	[0, 896]
	tai100b	100	1842	799	29	[0, 798]
	tai100c	100	2043	905	21	[0, 900]
	tai100d	100	1297	782	23	[0, 777]
	tai150a	150	1544	1062	30	[0, 1058]
	tai150b	150	1918	988	27	[0, 986]
	tai150c	150	2021	1081	23	[0, 1079]
	tai150d	150	1874	1025	26	[0, 1023]

Table 2
Characteristics of the dynamic delivery instances.

Type	Instance	N	Q	T	S	τ
Christofides	c50D	50	160	351	15	[0, 345]
	c75D	75	140	346	16	[0, 345]
	c100D	100	200	399	12	[0, 374]
	c100bD	100	200	468	12	[0, 439]
	c120D	120	200	794	13	[0, 792]
	c150D	150	200	399	10	[0, 397]
	c199D	199	200	399	9	[0, 392]
Fisher	f71D	71	30,000	211	5	[0, 208]
	f134D	134	2210	11,741	13	[0, 11700]
Taillard	tai75aD	75	1445	769	32	[0, 767]
	tai75bD	75	1679	905	26	[0, 903]
	tai75cD	75	1122	782	25	[0, 776]
	tai75dD	75	1699	789	27	[0, 787]
	tai100aD	100	1409	897	30	[0, 896]
	tai100bD	100	1842	799	29	[0, 798]
	tai100cD	100	2043	905	21	[0, 900]
	tai100dD	100	1297	782	23	[0, 777]
	tai150aD	150	1544	1062	30	[0, 1058]
	tai150bD	150	1918	988	27	[0, 986]
	tai150cD	150	2021	1081	23	[0, 1079]
	tai150dD	150	1874	1025	26	[0, 1023]

Table 3
Parameter of the DVRP model.

#	Parameter	Value	Description
1	n_{ts}	25	Number of time slices
2	t_{co}	0.5	Cutoff time
3	t_{ac}	0	Advanced commitment time
4	T	Max time available	Total length of working day

should be set to 25. This parameter divides the day T into 25 discrete time slices. The cutoff time t_{co} is equal to 0.5, indicating that half of the customers are considered static, while the remaining half are considered dynamic. The commitment time t_{ac} is assigned as 0, and the total length of working day T is equal to the maximum of time available section.

To determine the optimal value for the proposed approach's tunable parameter, the maximum num-ber of iterations, we randomly selected ten different instances to analyze their convergence behavior. The selected instances were c100b, c199D, tai100d, f71, tai150bD, tai75aD, tai100c, f134D, tai150a, and tai150c. For each instance, the proposed approach was executed 10 times independently, and only cus-tomers at time t=0 considered. The experimental results are presented in Fig. 9, where each point repre-sents the moment when the new best solution was found. As can be seen from the figure, most of the best solutions are found before 200 iterations. There-fore, we determined that 200 iterations are appropri-ate for this parameter.

4.3. Effectiveness evaluation

The performance of GRASP/VND is influenced by the construction methods used in the algorithm. In this study, we evaluated the traditional GRASP/VND using all construction methods as well as each construction method individually. The compared variants have the same random seeds, termination condition, number of runs and computer resource. Each variant is executed 10 times over the ten instances. The results of the experiment are presented in Table 4, with the best results highlighted in bold font. The table shows that using all construction methods in GRASP/VND leads to better search performance than using a single construction method. This is evident form the fact that GRASP/VND with all construction methods produced the best results for all instances, while each individual construction method performed well only for specific instances. Therefore, it can be concluded that the use of multiple construction methods in GRASP/VND is advantageous and should be considered when solving the VRPDR.

In the subsequent analysis, we explore the impact of neighborhood structures on the performance of the algorithms. Similar to the previous experiment, all algorithms (GRASP/VND, N_1, N_2, N_3, N_4, N_5, and N_6) were executed with the same parameters and resources to ensure a fair

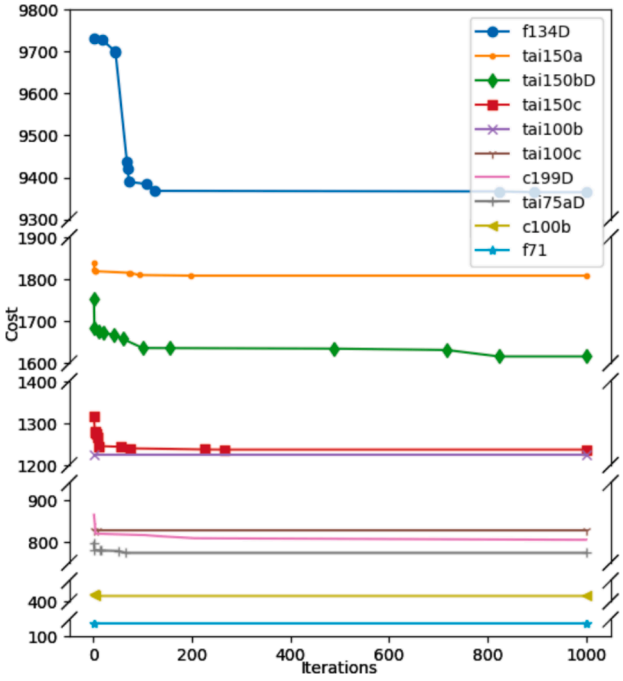

Fig. 9. Convergence of solution.

Table 4
Average results of GRASP/VND, Closest, Nearest, Farthest, and Earliest.

Instance	GRASP/VND	Closest	Nearest	Farthest	Earliest
c100b	**833.55**	871.31	854.35	878.56	847.39
c199D	**1515.36**	1591.41	1626.81	1626.62	1592.76
f134D	**12481.80**	15621.42	13808.26	15529.22	13538.66
f71	**253.64**	314.29	302.25	300.62	310.07
tai100b	**2066.47**	2269.96	2141.29	2199.73	2158.48
tai100c	**1459.31**	1624.54	1519.75	1589.38	1521.07
tai150a	**3187.69**	3334.59	3283.72	3339.14	3302.82
tai150bD	**2829.76**	2977.02	3010.28	2997.50	2956.94
tai150c	**2441.06**	2682.59	2537.22	2596.19	2518.44
tai75aD	**1713.52**	1786.80	1775.85	1787.13	1764.12

comparison. Additionally, each variant was executed 10 times on ten instances to obtain more reliable results. The results obtained from all variants are summarized in Table 5. It is evident from the table that GRASP/VND outperformed all other variants in every scenario. This suggests that the proposed components are effective in improving the search performance of the algorithm.

From Tables 4 and 5, The better performance achieved by GRASP/VND can be attributed to several characteristics. Firstly, during the construction phase, different construction methods are employed to generate initial solutions, which increases the diversity of the solution. This helps in avoiding premature convergence to local optima and encourages exploration of the search space. Secondly, VND explores different neighborhoods of the search space through the use of multiple neighborhood structures. By employing multiple neighborhood structures, VND is able to search the space in different ways in each iteration. This allows for a more comprehensive exploration of the search space and helps in finding better solutions. The combined approach of using different construction methods and multiple neighborhood structures enhances the overall search efficiency of the GRASP/VND algorithm. It enables the algorithm to explore a wider range of solutions and avoid getting stuck in local optima.

5. Algorithm comparisons

5.1. Comparison based on pickup instances

This section presents a comprehensive comparison of the proposed method with other algorithms to evaluate its effectiveness. Specifically, we analyzed the best and average results obtained by four algorithms, namely E-ACO [32], BSO [13], ACO-CD [12] and AAC [33]. We conducted 10 independent runs for each experiment and recorded the best and average results for each instance. Table 6 and 7 present the best and average values of this comparison, where "Instance" represents the test instance. We choose the evaluation function (9) to represent the gap in cost reduction between our algorithm and other algorithms.

$$Gap = \frac{f(ours) - f(others)}{f(ours)} \tag{9}$$

In Eq. (9) $f(ours)$ is the objective function obtained by the proposed algorithm, while $f(others)$ represents the outcomes achieved by other compared algorithms. If the gap is negative, it suggests that the proposed algorithm surpasses the other algorithms. Conversely, if the gap is positive, it implies that the proposed algorithm falls short compared to the result obtained by other algorithms. The values in bold are the best results among five algorithms

Base on a comprehensive analysis of Tables 6 and 7, it can be seen that GRAPS/VND achieved the optimal solution in 12 instances, while E-ACO, BSO, and AAC attained the best solution in 4, 0, and 5 instances respectively. Moreover, a detailed comparison of the best and average costs obtained from these methods has been presented. Notably, the total best cost obtained through GRASP/VND demonstrates a significant advantage over the other methods. Specifically, it yields a lower total

Table 5
Average results of GRASP/VND, N_1, N_2, N_3, N_4, N_5, and N_6.

Instance	GRASP/VND	N_1	N_2	N_3	N_4	N_5	N_6
c100b	**833.55**	1224.47	1190.58	1138.40	1133.62	913.80	1179.76
c199D	**1515.36**	2133.42	2340.63	2262.79	2086.55	1729.40	2173.53
f134D	**12481.80**	20529.15	21023.37	19888.97	20409.65	15152.19	21192.46
f71	**253.64**	366.91	415.18	355.08	379.08	332.73	404.09
tai100b	**2066.47**	2708.29	3010.38	2707.06	2770.10	2420.15	2899.11
tai100c	**1459.31**	1916.48	2193.87	1921.94	2010.11	1651.38	2303.33
tai150a	**3187.69**	3799.51	4565.30	3906.83	4053.15	3606.68	4542.18
tai150bD	**2829.76**	3692.07	4151.74	3669.08	3889.77	3206.94	4141.38
tai150c	**2441.06**	3367.08	4248.81	3319.36	3731.17	2997.41	4145.02
tai75aD	**1713.52**	2062.28	2361.81	2107.81	2208.47	1907.01	2314.37

Table 6
Best results and comparison with literature for dynamic pickup instances.

Instance	GRASP/VND	E-ACO		BSO		AAC	
		Best	Gap(%)	Best	Gap(%)	Best	Gap(%)
c50	**534.54**	607.21	−13.59	597.18	−11.72	551.95	−3.26
c75	931.51	**924.71**	0.73	938.79	−0.78	1156.83	−24.19
c100	**906.61**	973.40	−7.37	912.83	−0.69	1311.72	−44.68
c100b	833.55	869.22	−4.28	900.48	−8.03	**800.93**	3.91
c120	1307.59	1108.15	15.25	1173.10	10.29	**1049.47**	19.74
c150	**1211.50**	1378.63	−13.80	1254.47	−3.55	2188.33	−80.63
c199	**1519.62**	1561.12	−2.73	1631.23	−7.34	1650.85	−8.64
f71	**253.64**	259.71	−2.39	270.28	−6.56	301.79	−18.98
f134	**12821.86**	–	–	14831.29	−15.67	13015.56	−1.51
tai75a	**1710.31**	1690.91	1.13	1713.46	−0.18	1755.33	−2.63
tai75b	1373.51	1509.56	−9.91	1426.93	−3.89	**1306.47**	4.88
tai75c	1404.66	**1329.42**	5.36	1392.96	0.83	1424.76	−1.43
tai75d	1389.65	1409.14	−1.40	1503.79	−8.21	**1334.67**	3.96
tai100a	**2128.47**	2281.7	−7.20	2237.90	−5.14	2194.93	−3.12
tai100b	**2066.47**	2255.83	−9.16	2068.12	−0.08	2126.09	−2.89
tai100c	1459.31	**1442.45**	1.16	1479.48	−1.38	1544.50	−5.84
tai100d	1747.55	**1581.36**	9.51	1855.26	−6.16	1909.55	−9.27
tai150a	3187.69	3307.63	−3.76	3391.39	−6.39	**2999.27**	5.91
tai150b	**2799.55**	3128.00	−11.73	2899.56	−3.57	2846.28	−1.67
tai150c	**2441.06**	2583.36	−5.83	2596.67	−6.37	2718.36	−11.36
tai150d	**2757.04**	2808.99	−1.88	3073.54	−11.48	3230.67	−17.18
			−3.10		−4.58		−9.47

Table 7
Average results and comparison with literature for dynamic pickup instances.

Instance	GRASP/VND	E-ACO		ACO-CD		AAC	
		Avg	Gap(%)	Avg	Gap(%)	Avg	Gap(%)
c50	577.70	647.21	−12.03	593.32	−2.70	**570.89**	1.18
c75	**955.52**	1045.44	−9.41	944.18	1.19	1213.45	−26.99
c100	**955.87**	1044.96	−9.32	963.80	−0.83	1380.25	−44.40
c100b	855.90	950.17	−11.01	948.56	−10.83	**841.44**	1.69
c120	1359.79	1197.68	11.92	1235.30	9.16	**1153.29**	15.19
c150	**1265.82**	1472.40	−16.32	1268.16	−0.18	2386.93	−88.57
c199	**1562.66**	1836.86	−17.55	1566.37	−0.24	1758.51	−12.53
f71	294.61	297.08	−0.84	**289.24**	1.82	309.94	−5.20
f134	**13563.43**	–	–	15827.69	−16.69	15528.81	−14.49
tai75a	**1765.18**	1983.92	−12.39	1964.59	−11.30	1782.91	−1.00
tai75b	**1395.83**	1647.78	−18.05	1527.33	−9.42	1452.26	−4.04
tai75c	1479.00	1470.6	0.57	1617.84	−9.39	**1441.91**	2.51
tai75d	**1407.26**	1661.73	−18.08	1522.00	−8.15	1422.27	−1.07
tai100a	**2192.67**	2550.64	−16.33	2213.04	−0.93	2232.71	−1.83
tai100b	**2142.63**	2500.72	−16.71	2391.59	−11.62	2182.61	−1.87
tai100c	**1493.52**	1743.07	−16.71	1584.20	−6.07	1562.66	−4.63
tai100d	**1829.41**	1843.82	−0.79	2275.15	−24.37	1912.43	−4.54
tai150a	3244.14	3684.03	−13.56	3473.58	−7.07	**3185.73**	1.80
tai150b	**2871.59**	3439.38	−19.77	3347.30	−16.57	2880.57	−0.31
tai150c	**2505.24**	2729.15	−8.94	2851.53	−13.82	2743.55	−9.51
tai150d	**2817.62**	3186.08	−13.08	3191.28	−13.26	3345.16	−18.72
			−10.92		−7.20		−10.35

best cost compared to E-ACO, BSO, and AAC, with percentage gaps of 3.10 %, 4.58 %, and 9.47 % respectively. Furthermore, when considering the average total cost, GRASP/VND consistently outperforms the other methods by obtaining solutions that are 10.92 % shorter (better) than E-ACO, 7.20 % shorter than ACO-CD, and 10.35 % shorter than AAC. Finally, we have presented 15 new improved solutions.

In Table 8, the computational time (in seconds) for GRASP/VND and AAC is reported. It should be noted that the computational time for E-AC and ACO-CD is not provided, hence they are excluded from the comparison. The data shows that, for all instances, the GRASP/VND algorithm exhibits significantly lower computational time compared to the AAC algorithm. Specifically, the average computational time for GRASP/VND is 346.66 s, whereas for AAC it is 2140.63 s, indicating that GRASP/VND is approximately 6 times faster on average. The results suggests that the GRASP/VND provides a significantly more efficient approach for solving the VRPDR.

5.2. Comparison based on delivery instances

Although most research on dynamic vehicle routing problem adopt the pickup instances, this paper will add the delivery instances to our experiment. We selected the ContDVRP algorithm from Okulewicz [34] for the comparative experiment, and the program code can be found at https://sourceforge.net/projects/continuous-dvrp/.

An analysis of Table 9 reveals that the GRASP/ VND was effective in finding 11 new best solutions out of 21 instances. The gap between those solutions and the previously established best solutions falls within a range of −9.88 % to −0.19 %. These solutions exhibit a high level of quality, indicating the strength of the proposed algorithm in optimizing DVRP solutions. It is noteworthy that, among the remaining 10 solutions obtained through the GRASP/VND algorithm, they do not demonstrate significant advantages. The gap between these solutions varies from 0.85 % to 14.76 %, which can be attributed, in part, to differences in the algorithmic approach. In contrast, the ContDVRP utilizes a parallel processing method by assuming that the problem is solved in a parallel way on a single multithreading computer. When compared to ContDVRP, our proposed algorithm exhibits a gap of 0.51 and 1.21 between the best and average results, respectively. These findings underscore the effectiveness of the proposed algorithm in generating high-quality solutions for the DVRPs. Overall, these results demonstrate the GRASP/VND algorithm's potential in tackling DVRP problems and contribute to advancing our understanding of these complex logistical challenges.

Table 10 presents a comparison of the computational time for the GRASP/VND algorithm and the ContDVRP algorithm. It is worth noting that the ContDVRP algorithm utilizes parallel optimization during execution. From the data in the table, it can be observed that, on average, the GRASP/VND algorithm requires 343.94 s, slightly longer than the ContDVRP algorithm's 228.36 s. However, when analyzing from an individual instance perspective, it can be observed that except for a significant difference in the c199D instance, the difference in computational time between the two algorithms is not large overall. Further examination reveals that the GRASP/VND algorithm outperforms the ContDVRP algorithm in terms of computational time on 7 instances. In conclusion, although the GRASP/VND algorithm slightly lags behind the ContDVRP algorithm with parallel optimization in terms of average computational time, they are comparable when considering individual instances, and the GRASP/VND algorithm demonstrates superior computational efficiency on multiple instances. Therefore, it can be concluded that the GRASP/VND algorithm is effective in solving the VRPOD problem.

Table 8
Computational time of GRASP/VND compared with AAC.

Instance	GRASP/VND	AAC
c50	**39.92**	525.68
c75	**81.05**	1142.41
c100	**251.54**	1356.50
c100b	**158.37**	2086.25
c120	**420.41**	2471.25
c150	**812.37**	3861.15
c199	**1565.87**	3001.35
f71	**103.25**	1143.72
f134	**283.28**	3417.7
tai75a	**64.30**	942.42
tai75b	**59.96**	906.60
tai75c	**76.24**	906.20
tai75d	**65.33**	1091.89
tai100a	**233.45**	1917.05
tai100b	**175.45**	1887.13
tai100c	**147.30**	1929.09
tai100d	**191.10**	1991.84
tai150a	**826.17**	3323.27
tai150b	**705.41**	3687.81
tai150c	**559.37**	3867.34
tai150d	**459.76**	3496.64
Avg	**346.66**	2140.63

6. Conclusion

This paper presents a hybrid metaheuristic approach for solving the Vehicle Routing Problem with Dynamic Requests (VRPDR), which effectively integrates the Greedy Randomized Adaptive Search Procedure (GRASP) and Variable Neighborhood Descent (VND) search processes. The proposed method leverages the strengths of both GRASP and VND, initiating with GRASP to construct an initial solution and subsequently employing VND to explore new search areas and refine the solutions, thereby enhancing the quality of the results towards optimality. The hybrid approach is enriched by various construction algorithms and neighborhood structures, which contribute to its robustness and efficiency.

Evaluation on modified VRP benchmarks demonstrates that our GRASP/VND algorithm outperforms five other algorithms in multiple instances, highlighting its advantages in both convergence and diversity. While these results indicate that the hybrid approach is a viable and competitive solution for the VRPDR, it is important to acknowledge certain limitations. For instance, the current implementation does not consider some realistic constraints such as vehicle limitations and personnel types, which could impact its applicability in certain practical scenarios.

To address these limitations and enhance the real-world applicability of our approach, future work will focus on incorporating additional realistic constraints. This will not only increase the realism of the problem but also broaden the scope of potential applications, making the research more relevant to practitioners in the field of logistics optimization. Furthermore, exploring the integration of other advanced metaheuristics or machine learning techniques could provide even more efficient and effective solutions for the VRPDR. Overall, this study significantly contributes to the field of logistics optimization by presenting a competitive and adaptable hybrid approach for solving the VRPDR.

Funding

This work was supported by the Scientific Research Projects of Ordinary Universities in Guangdong Province-Young Innovative Talents Projects (2019KQNCX073), Technology Innovation Strategy Fund Project of Guangdong Province [2023A01021,2023A01025, and pdjh2024a237], General project of Natural Science Foundation of Guangdong Province (2023A1515011568 and 2024A1515011674).

CRediT authorship contribution statement

Shifeng Chen: Writing – review & editing, Project administration.

Table 9
Comparison with literature for dynamic delivery instances.

Instance	GRASP/VND		ContDVRP			
	Best	Avg	Best	Gap(%)	Avg	Gap(%)
C50D	548.44	582.42	**532.10**	2.98	**573.76**	1.49
C75D	906.14	959.88	**893.66**	1.38	**907.81**	5.42
C100D	**901.74**	941.08	903.48	−0.19	**931.59**	1.01
C100bD	828.65	**844.95**	**821.63**	0.85	858.29	−1.58
C120D	1253.93	1358.27	**1068.85**	14.76	**1120.19**	17.53
C150D	1186.12	1226.57	**1105.03**	6.84	**1146.23**	6.55
C199D	1515.36	1562.88	**1393.25**	8.06	**1437.71**	8.01
f71D	**263.00**	282.75	271.86	−3.37	**277.45**	1.87
f134D	12481.80	13134.04	**11730.88**	6.02	**11857.85**	9.72
tai75aD	**1713.52**	**1746.43**	1741.37	−1.63	1796.39	−2.86
tai75bD	**1378.48**	**1390.53**	1385.43	−0.50	1397.50	−0.50
tai75cD	**1407.74**	1495.82	1419.08	−0.81	**1486.38**	0.63
tai75dD	**1412.57**	**1431.84**	1415.51	−0.21	1432.58	−0.05
tai100aD	**2112.62**	**2167.43**	2176.48	−3.02	2206.81	−1.82
tai100bD	2073.00	2150.86	**2054.26**	0.90	**2099.68**	2.38
tai100cD	1484.43	10205513.17	**1458.75**	1.73	**1484.76**	1.88
tai100dD	1724.44	1775.11	**1663.20**	3.55	**1710.36**	3.65
tai150aD	**3190.20**	**3246.98**	3505.42	−9.88	3570.71	−9.97
tai150bD	**2829.76**	**2926.03**	2931.69	−3.60	2999.52	−2.51
tai150cD	**2419.45**	**2489.02**	2612.93	−8.00	2679.78	−7.66
tai150dD	**2750.79**	**2807.83**	2891.39	−5.11	3027.02	−7.81
				0.51		

Table 10
Computational time of GRASP/VND compared with ContDVRP.

Instance	GRASP/VND	ContDVRP
c50D	**40.56**	45.02
c75D	**85.47**	94.95
c100D	201.81	91.63
c100bD	157.98	117.03
c120D	426.89	137.74
c150D	834.58	304.13
c199D	1544.40	683.79
f71D	107.78	52.59
f134D	**191.09**	232.98
tai75aD	**85.42**	92.87
tai75bD	**59.81**	99.01
tai75cD	**69.27**	88.82
tai75dD	114.43	99.42
tai100aD	173.68	128.55
tai100bD	168.04	130.92
tai100cD	**142.91**	160.28
tai100dD	231.08	151.57
tai150aD	847.36	522.07
tai150bD	728.30	522.88
tai150cD	563.38	562.44
tai150dD	**448.56**	476.78
Avg	343.94	228.36

Yanlan Yin: Validation, Investigation, Formal analysis. **Haitao Sang:** Writing – review & editing, Investigation, Formal analysis. **Wu Deng:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Dantzig GB, Ramser JH. The truck dispatching problem. Manag Sci 1959;6:80–91.
[2] Chen C, Demir E, Huang Y. An adaptive large neighborhood search heuristic for the vehicle routing problem with time windows and delivery robots. Eur J Oper Res 2021;294:1164–80.
[3] Yuan Y, Cattaruzza D, Ogier M, Semet F, Vigo D. A column generation based heuristic for the generalized vehicle routing problem with time windows. Transp Res Part E 2021;152:102391.
[4] Hesam Sadati ME, Çatay B, Aksen D. An efficient variable neighborhood search with tabu shaking for a class of multi-depot vehicle routing problems. Comput Oper Res 2021;133:105269.
[5] Moonsri K, Sethanan K, Worasan K, Nitisiri K. A hybrid and self-adaptive differential evolution algorithm for the multi-depot vehicle routing problem in egg distribution. Appl Sci 2021;12:35.
[6] Pan B, Zhang Z, Lim A. A hybrid algorithm for time-dependent vehicle routing problem with time windows. Comput Oper Res 2021;128:105193.
[7] Gmira M, Gendreau M, Lodi A, Potvin J-Y. Tabu search for the time-dependent vehicle routing problem with time windows on a road network. Eur J Oper Res 2021;288:129–40.
[8] Basso R, Kulcsár B, Sanchez-Diaz I. Electric vehicle routing problem with machine learning for energy prediction. Transp Res B Methodol 2021;145:24–55.
[9] Kucukoglu I, Dewil R, Cattrysse D. The electric vehicle routing problem and its variations: a literature review. Comput Ind Eng 2021;161:107650.
[10] Asghari M, Mirzapour Al-e-hashem SMJ. Green vehicle routing problem: a state-of-the-art review. Int J Prod Econ 2021;231:107899.
[11] Moghdani R, Salimifard K, Demir E, Benyettou A. The green vehicle routing problem: a systematic literature review. J Clean Prod 2021;279:123691.
[12] Xiang X, Qiu J, Xiao J, Zhang X. Demand coverage diversity based ant colony optimization for dynamic vehicle routing problems. Eng Appl Artif Intel 2020;91:103582.
[13] Liu M, Shen Y, Shi Y. A hybrid brain storm optimization algorithm for dynamic vehicle routing problem. In: Tan Y, Shi Y, Tuba M, editors. Advances in Swarm Intelligence. Cham: Springer International Publishing; 2020. p. 251–8.
[14] Chen S, Chen R, Wang G-G, Gao J, Sangaiah AK. An adaptive large neighborhood search heuristic for dynamic vehicle routing problems. Comput Electr Eng 2018;67:596–607.
[15] Psaraftis HN. A dynamic programming solution to the single vehicle many-to-many immediate request dial-a-ride problem. Transp Sci 1980;14:130–54.
[16] Kilby P, Prosser P, Shaw P. Dynamic VRPs: a study of scenarios, University of Strathclyde. Tech Rep 1998;1.
[17] Montemanni R, Gambardella LM, Rizzoli AE, Donati AV. Ant colony system for a dynamic vehicle routing problem. J Comb Optim 2005;10:327–43.
[18] Khouadjia MR, Sarasola B, Alba E, Jourdan L, Talbi EG. A comparative study between dynamic adapted PSO and VNS for the vehicle routing problem with dynamic requests. Appl Soft Comput J 2012;12:1426–39.
[19] Mańdziuk J, Żychowski A. A memetic approach to vehicle routing problem with dynamic requests. Appl Soft Comput 2016;48:522–34.
[20] Yi R, Luo W, Bu C, Lin X. A hybrid genetic algorithm for vehicle routing problems with dynamic requests. In: 2017 IEEE Symposium Series on Computational Intelligence (SSCI). Honolulu, HI: IEEE; 2017. p. 1–8.
[21] Abdirad M, Krishnan K, Gupta D. A two-stage metaheuristic algorithm for the dynamic vehicle routing problem in Industry 4.0 approach. J Manage Anal 2021;8:69–83.
[22] Lan L, Van Doorn JMH, Wouda NA, Rijal A, Bhulai S. An iterative sample scenario approach for the dynamic dispatch waves problem. Transp Sci 2024. trsc.2023.0111.
[23] Zhou Y, Kong L, Yan L, Liu Y, Wang H. A memetic algorithm for a real-world dynamic pickup and delivery problem. Memetic Comp 2024.
[24] Abdallah AMFM, Essam DL, Sarker RA. On solving periodic re-optimization dynamic vehicle routing problems. Appl Soft Comput 2017;55:1–12.
[25] Ren X, Chen S, Ren L. Optimization of regional emergency supplies distribution vehicle route with dynamic real-time demand. MBE 2023;20:7487–518.

[26] Gil-Borrás S, Pardo EG, Alonso-Ayuso A, Duarte A. GRASP with variable neighborhood descent for the online order batching problem. J Glob Optim 2020; 78:295–325.
[27] Barrero L, Robledo F, Romero P, Viera R. A GRASP/VND heuristic for the heterogeneous fleet vehicle routing problem with time windows. In: Mladenovic N, Sleptchenko A, Sifaleras A, Omar M, editors. Variable Neighborhood Search. Cham: Springer International Publishing; 2021. p. 152–65.
[28] Ronconi DP, Manguino JLV. GRASP and VNS approaches for a vehicle routing problem with step cost functions. Ann Oper Res 2022.
[29] Kyriakakis NA, Aronis S, Marinaki M, Marinakis Y. A GRASP/VND algorithm for the energy minimizing drone routing problem with pickups and deliveries. Comput Ind Eng 2023;182:109340.
[30] Feo TA, Resende MGC. A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 1989;8:67–71.
[31] Mladenovi N, Hansen P. Variable neighborhood search. Comput Oper Res 1997;24: 1097–100.
[32] Xu H, Pu P, Duan F. Dynamic vehicle routing problems with enhanced ant colony optimization. Discret Dyn Nat Soc 2018;2018:1–13.
[33] Euchi J, Yassine A, Chabchoub H. The dynamic vehicle routing problem: Solution with hybrid metaheuristic approach. Swarm Evol Comput 2015;21:41–53.
[34] Okulewicz M, Mandziuk J. Two-phase multi-swarm PSO and the dynamic vehicle routing problem. In: 2014 IEEE Symposium on Computational Intelligence for Human-like Intelligence (CIHLI). Orlando, FL, USA: IEEE; 2014. p. 1–8.